МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МИРЭА – РОССИЙСКИЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»

РТУ МИРЭА
КОЛЛЕДЖ ПРОГРАММИРОВАНИЯ И КИБЕРБЕЗОПАСНОСТИ

ДОКЛАД

По проекту
«Разработка информационной системы для ООО «Обувь»»

по МДК.01.01 Разработка программных модулей
Специальность
09.02.07 Информационные системы и программирование

Выполнили
Студенты группы ЩПКО-03-23
Караулов С. Моховая А. Зайнетдинова Р.

Проверил преподаватель
Стоколос М. Д.

Москва 2025

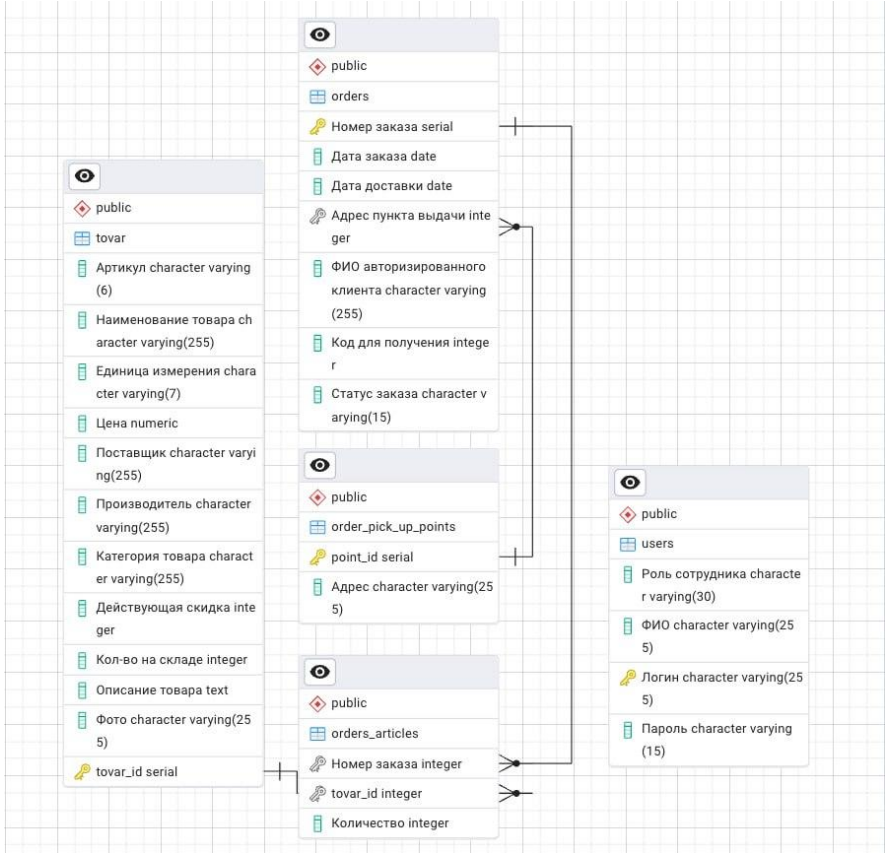
Постановка задачи и хода работы:

Цель: Создание настольного приложения для управления товарами и заказами для магазина ООО «Обувь».

Ход работы:

- База данных — спроектировать БД, загрузить данные, выгрузить ER-диаграмму и скрипт.
- Приложение — сделать вход по ролям, вывод товаров с подсветкой.
- Товары — добавить поиск, фильтр, сортировку и форму редактирования для админа.
- Заказы — реализовать просмотр и управление заказами для менеджера и админа.

Модуль 1. Проектирование Базы Данных



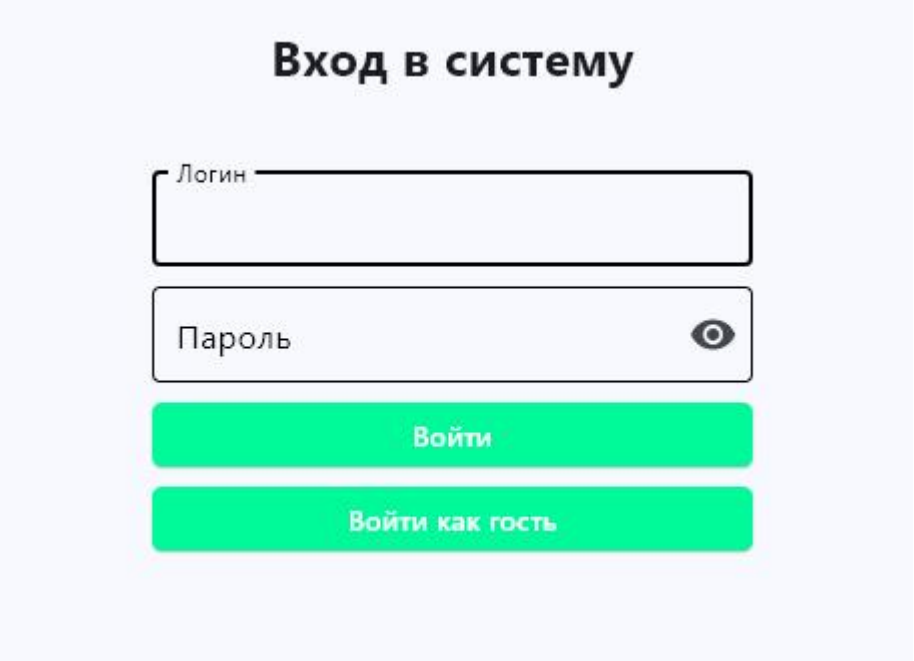
ER-диаграмма базы данных

Модуль 1. Проектирование Базы Данных

```
class Database:
    def init(self):
        self.connection = None

    def connect(self):
        try:
            self.connection = psycopg2.connect(**config.DB_CONFIG)
            return True
        except Exception as e:
            print(f"Ошибка подключения к БД: {e}")
            return False

    def execute_query(self, query, params=None):
        try:
            with self.connection.cursor(cursor_factory=RealDictCursor) as cursor:
                cursor.execute(query, params)
                if query.strip().upper().startswith('SELECT') or ('RETURNING' in query.upper()):
                    return cursor.fetchall()
                else:
                    self.connection.commit()
                    return cursor.rowcount
        except Exception as e:
            print(f"Ошибка выполнения запроса: {e}")
            return None
```

Скрипт для взаимодействия с базой данных

Модуль 2. Авторизация



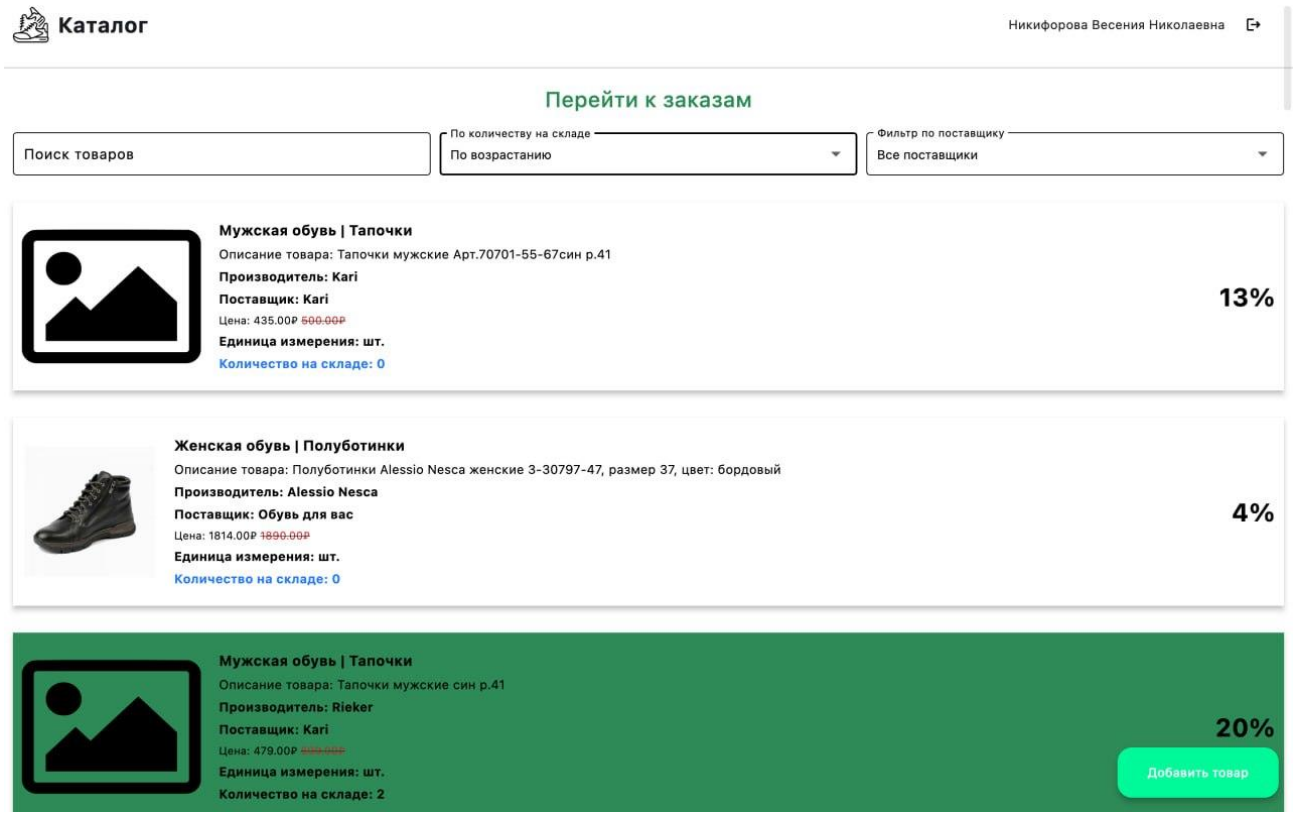
Окно авторизации

Модуль 2. Авторизация

```
def login_click(self, e):
    """Обработка входа по логину/паролю"""
    try:
        # 1. Получение данных из полей ввода
        username = self.username_field.value.strip()
        password = self.password_field.value

        # 2. Базовая валидация входных данных
        if not username:
            DialogManager.show_error_dialog(self.page, "Ошибка входа", "Пожалуйста, введите логин.")
            return
        if not password:
            DialogManager.show_error_dialog(self.page, "Ошибка входа", "Пожалуйста, введите пароль.")
            return

        # 3. Проверка учетных данных в базе данных
        user_data = self.db.get_user_with_credentials(username, password)

        if not user_data:
            DialogManager.show_error_dialog(self.page, "Ошибка входа", "Неверный логин или пароль.")
            self.password_field.value = "" # Очистка поля пароля
            self.page.update()
            return

        # 4. Создание объекта пользователя и успешный вход
        user = User(user_data)
        DialogManager.show_success_dialog(
            self.page,
            "Успешный вход",
            f"Добро пожаловать, {user.name}!",
            on_close=lambda: self.navigate_to_main_app(user) # Перенаправление
        )
```

Функция получения данных обработки результата

Модуль 2. Список товаров и Интерфейс



Главное окно приложения

Модуль 2. Список товаров

Функция отображения сетки товаров

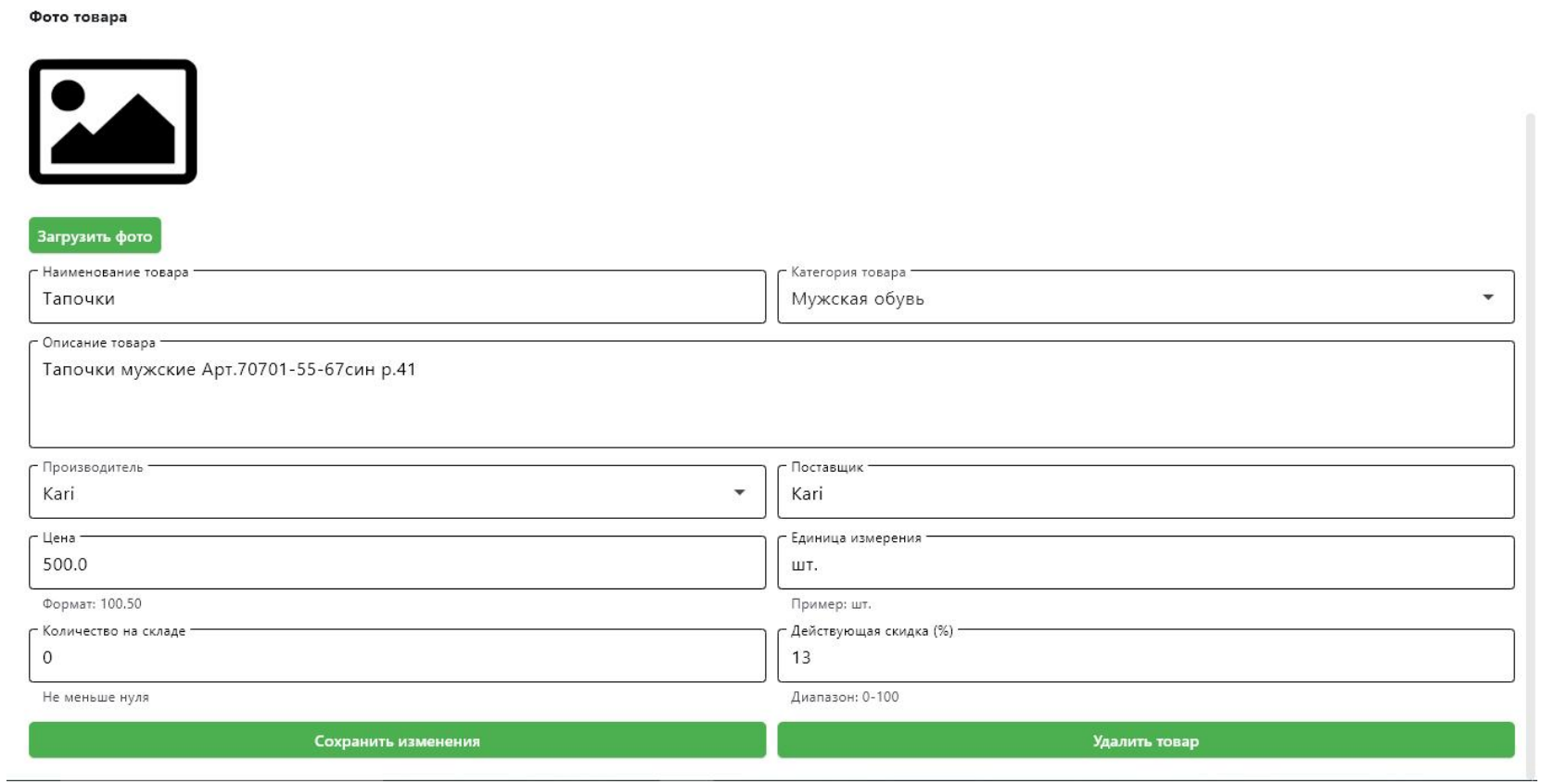
```
def create_products_grid(self):
        """Создание сетки товаров"""
        if not self.products:
            return ft.Text("Товары не найдены")

        product_cards = []
        if not self.filtered_products:
            product_cards.append(
                ft.Container(
                    content=ft.Text("Товары не найдены", size=16),
                    alignment=ft.alignment.center,
                    padding=20
                )
            )
        else:
            for product in self.filtered_products:
                card = self.create_product_card(product)
                product_cards.append(card)

        return ft.Column(
            ref=self.products_grid_ref,
            controls=product_cards,
            spacing=10,
            run_spacing=10,
        )
```

Модуль 3. Фильтрация и Сортировка

Функция фильтрации и поиска товаров

```
def filter_products(self, e=None):
        """Фильтрация товаров и поиск"""
        self.search_query = e.control.value if e else self.search_query
        self.filtered_products = self.products.copy()

        if self.search_query:
            search_words = self.search_query.lower().split()

            self.filtered_products = [
                product for product in self.products
                if all(
                    any(
                        word in (getattr(product, field) or "").lower()
                        for field in ['name', 'category', 'manufacturer', 'supplier', 'description', 'article']
                    )
                    for word in search_words
                )
            ]

        if self.filter_by_supplier != "Все поставщики":
            self.filtered_products = [
                p for p in self.filtered_products
                if p.supplier == self.filter_by_supplier
            ]

        if self.sort_by_stock == "По возрастанию":
            self.filtered_products.sort(key=lambda p: p.stock)
        elif self.sort_by_stock == "По убыванию":
            self.filtered_products.sort(key=lambda p: p.stock, reverse=True)

        self.update_products_grid()
        print(*[p.name for p in self.filtered_products])
```

Модуль 3. Редактирование товаров



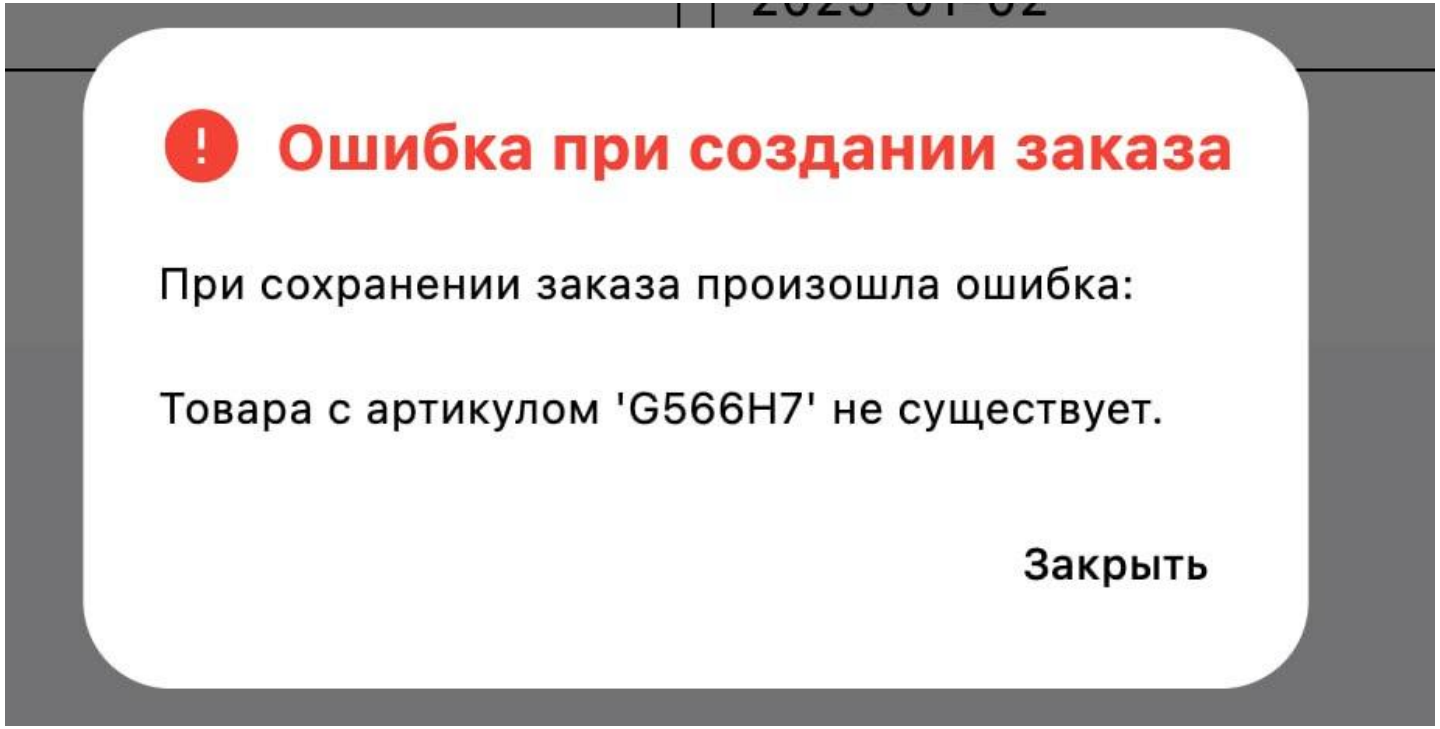
Окно редактирование товара

Модуль 3. Редактирование товаров

Обновление данных о товаре

```
# Обновление товара в базе данных
self.db.update_product(
    tovar_id=self.tovar_id,
    name=self.name_input.value,
    unit=self.unit_input.value,
    price=float(self.price_input.value),
    supplier=self.supplier_input.value,
    manufacturer=self.manufacturer_dropdown.value,
    category=self.category_dropdown.value,
    discount=float(self.discount_input.value) if
self.discount_input.value else 0,
    stock=int(self.stock_input.value),
    description=self.description_input.value,
    photo_path=self.photo_path
)

DialogManager.show_success_dialog(
    self.page,
    "Сохранено",
    "Изменения успешно сохранены.",
    on_close=lambda: self.page.go("/catalog")
)
```

Модуль 3. Валидация полей



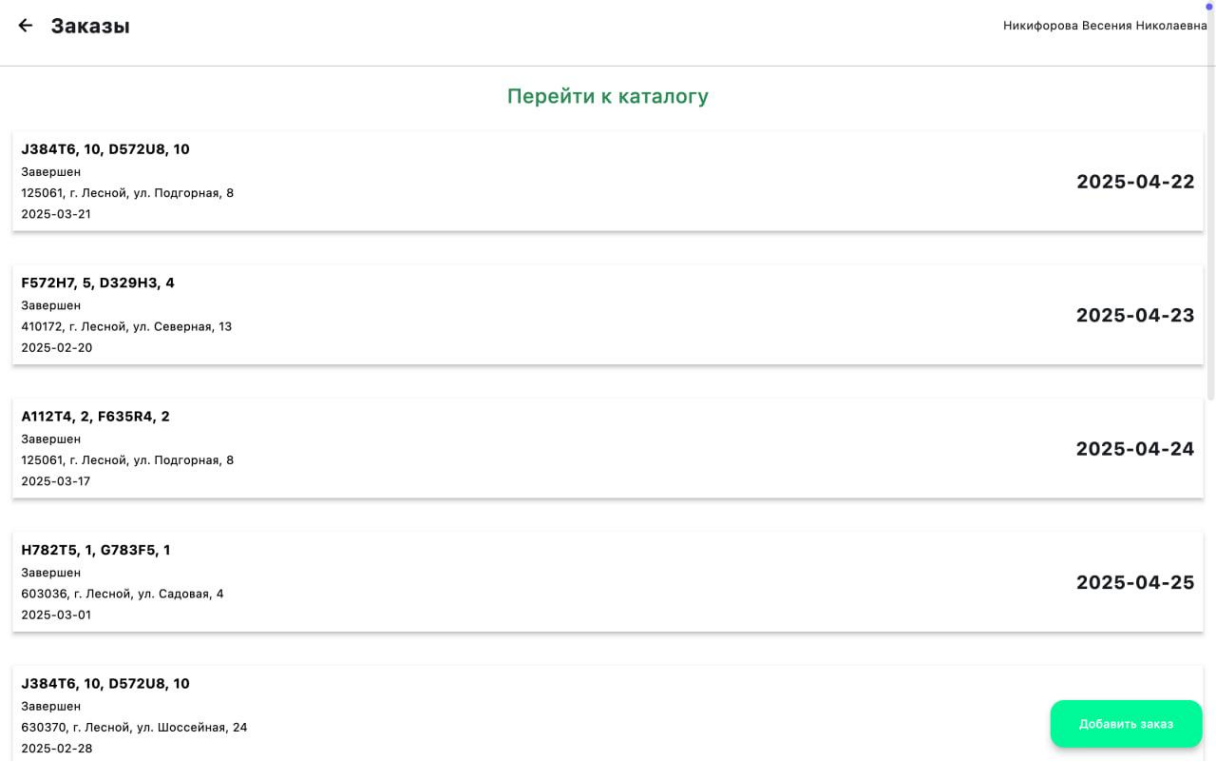
Окно предупреждения/ошибки

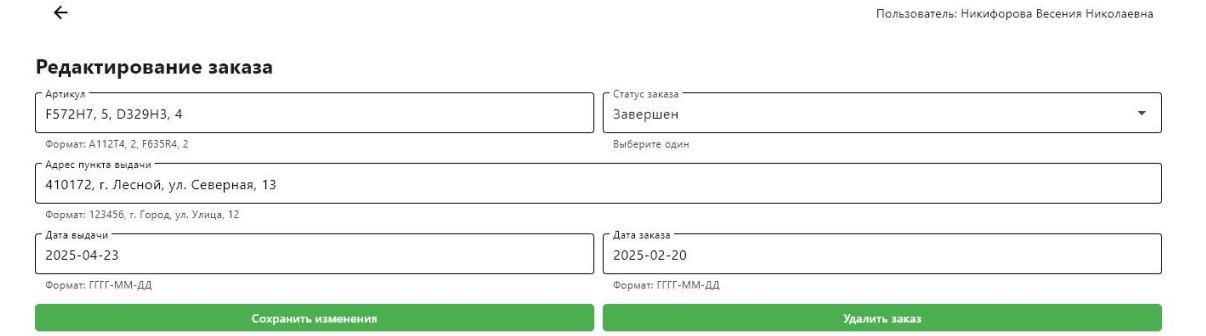
Модуль 3. Валидация полей

Функция обработки даных

```
def create_product(self, e):
    """Создание нового товара"""
    try:

        if not self.category_dropdown.value:
            DialogManager.show_error_dialog(
                self.page,
                "Ошибка валидации",
                "Выберите категорию товара."
            )
            return

        if not self.manufacturer_dropdown.value:
            DialogManager.show_error_dialog(
                self.page,
                "Ошибка валидации",
                "Выберите производителя товара."
            )
            return

        if not self.validate_inputs():
            DialogManager.show_error_dialog(
                self.page,
                "Ошибка валидации",
                "Пожалуйста, проверьте заполнение всех полей."
            )
            return
```

Модуль 4. Работа с заказами





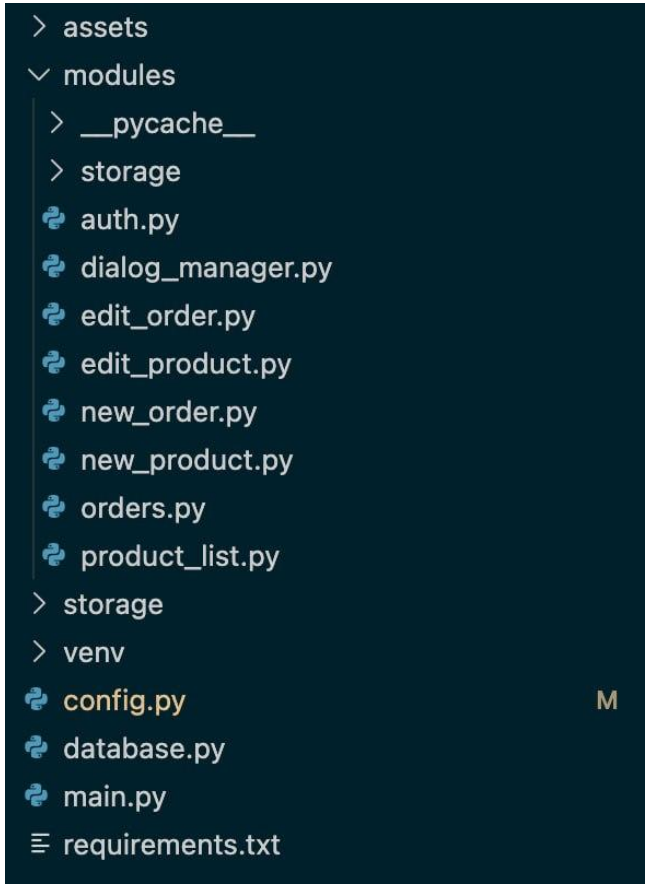
Окно заказов, редактирования/добавление заказов

Модуль 4. Работа с заказами

Функция создания заказа с валидацией полей

```
def create_order(self, e):
        """Создание нового заказа с полной обработкой ошибок"""
        try:
            if not self.status_drop_down.value:
                DialogManager.show_error_dialog(
                    self.page,
                    "Ошибка валидации",
                    "Выберите статус заказа."
                )
                return

            if not self.address_input.value or self.address_input.value.strip() == '':
                DialogManager.show_error_dialog(
                    self.page,
                    "Ошибка валидации",
                    "Укажите адрес пункта выдачи."
                )
                return

            if not self.date_deliver.value or self.date_deliver.value.strip() == '':
                DialogManager.show_error_dialog(
                    self.page,
                    "Ошибка валидации",
                    "Укажите дату доставки."
                )
                return

            if not self.date_order.value or self.date_order.value.strip() == '':
                DialogManager.show_error_dialog(
                    self.page,
                    "Ошибка валидации",
                    "Укажите дату заказа."
                )
                return
```

Структура проекта




```python
import os

DB_CONFIG = {
    'host': 'localhost',
    'database': 'shoe_store',
    'user': 'postgres',
    'password': "Пароль от БД'',
    'port': '5432'
}

BASE_DIR = os.path.dirname(os.path.abspath(file))
ASSETS_DIR = os.path.join(BASE_DIR, 'assets')
IMAGES_DIR = os.path.join(ASSETS_DIR, 'images')
TOVAR_IMAGES_DIR = os.path.join(IMAGES_DIR, 'tovar')

APP_TITLE = "ООО «Обувь» - Магазин обуви"
APP_ICON = os.path.join(IMAGES_DIR, "icon.ico")
PLACEHOLDER_IMAGE = os.path.join(IMAGES_DIR, "picture.png")
LOGO_IMAGE = os.path.join(IMAGES_DIR, "icon.png")
```

Файл конфигурации и структура проекта

Заключение

- Спроектирована и реализована база данных.
- Создано приложение с ролевым доступом.
- Реализован просмотр товаров с фильтрацией и поиском.
- Добавлены CRUD-операции для товаров и заказов.
- Обеспечена навигация и обработка ошибок.
- Система готова к работе.